

Arcade

Luuk Teunissen

Problem Analysis

1. Problem Description

The goal of this project is to design and develop a modular retro-style arcade fighting game. The player controls a character who must fight against a series of opponents. After an opponent is defeated, a new opponent spawns. This continues until the player runs out of lives.

2. Technical Approach

the game uses:

- reusable C++ code
- a modular game engine using object-oriented principles
- Interface with real hardware via a Hardware Abstraction Layer (HAL)

Requirements

1.1 Requirements

ID	REQUIREMENT	PRIORITY
FR-01	The game shall open on a start menu when launched.	SHOULD
FR-02	The game shall allow the player to control the character using the arrow keys and designated action keys .	MUST
FR-03	The character shall be able to move left and right .	
FR-04	The character shall be able to jump .	
FR-05	The character shall be able to duck .	
FR-06	The character shall be able to perform a kick attack .	
FR-07	The character shall be able to perform a punch attack .	
FR-08	The character shall be able to block incoming attacks .	

FR-09	The game shall be able to spawn enemies during gameplay.
FR-10	When an enemy is defeated, the game shall spawn a new enemy .
FR-11	The character has a health value that decreases whenever the character takes damage
FR-12	The character shall die when their health reaches zero .
FR-13	When the character dies, the game shall display a game over screen .
FR-14	The game over screen shall provide the player with the option to restart the game or quit .
FR-15	The game shall provide an interface with real hardware through a Hardware Abstraction Layer (HAL) .

1.2 nonfunctional requirements

ID	NONFUNCTIONAL REQUIREMENT	PRIORITY
NFR-01	The game shall be developed using C++ .	
NFR-02	The game shall be developed using a modular game engine architecture , allowing individual components to be developed and maintained independently.	
NFR-03	The game shall follow object-oriented programming principles in its design and implementation.	
NFR-04	The game's hardware interface shall use a Hardware Abstraction Layer (HAL) to separate hardware-specific functionality from the rest of the game software.	

2 task decomposition

system	info	
C++ UI	This UI makes it clear what things are on the screen, where they are, and what actions you, as a player, should perform.	
Audio handeler	Based on the players actions. Sound effects and music plays accordingly.	
Enemy design	Spawns an enemy that can preform	

Movement handler	This lets a player move within a space	
Action handler	Lets a player preform actions	

3 usecases

Use case 1	Start game
Actor(s):	Player
Trigger:	The player presses start
Happy Flow:	1 – the player presses (N.Y.D.) to start the game
Alternative	1 – the player presses nothing, so we remain on the startup screen
Objects/classes identified:	1- Player – interacts with the game and can press the start button. 2- Game – controls whether the game has started or is still on the startup screen. 3- StartupScreen – displays the screen shown before the game starts. 4- Input/Button – detects when the player presses the button/key used to start the game.
Possible responseibilities	1- Player → presses the start button. 2- Input/Button → detects the player's input. 3- Game → starts the game when the correct input is received. 4- StartupScreen → remains visible when the player has not pressed anything.

Use case 2	Movement
Actor(s):	Player
Trigger:	The player presses an arrow key or a designated action key.
Happy Flow:	1 – The player presses an arrow key or designated action key. 2 – The game detects the input. 3 – The character moves or performs the corresponding action.
Alternative	1- The player does not press any key. 2- The character remains in its current state.
Objects/classes identified:	1- Player – controls the character using the keyboard. 2- Character – responds to the player's input by moving or performing actions.

	3- Input/Keyboard – detects the arrow keys and designated action keys. 4- Game – processes the input and controls the interaction between the player and character.
Possible responseibilities	1- Player → provides input. 2- Input/Keyboard → detects which key was pressed. 3- Character → moves or performs an action. 4- Game → processes the input and applies it to the character.

Use case 3	Preform an Action
Actor(s):	Player
Trigger:	The player presses a designated action key.
Happy Flow:	1 – The player presses a designated action key. 2 – The game detects the input. 3 – The character performs the corresponding action.
Alternative	1 – The player does not press an action key. 2 – The character continues its current state.
Objects/classes identified:	1- Player – provides the input to perform an action. 2- Character – performs an action based on the player's input. 3- Game – handles the player's input and controls the game. 4- Input/Keyboard – detects the designated action keys.
Possible responseibilities	1- Player → provides input. 2- Input/Keyboard → detects which action key was pressed. 3- Character → performs the corresponding action. 4- Game → processes the input and tells the character which action to perform.

Use case 4	Block an Attack
Actor(s):	Player, enemies
Trigger:	The enemy attacks the character.
Happy Flow:	1 – The enemy attacks the character. 2 – The player presses the designated block key. 3 – The character blocks the attack. 4 – The character takes no damage.
Alternative	1 – The enemy attacks the character. 2 – The player does not block the attack in time. 3 – The character is hit by the attack.

	4 – The character takes damage.
Objects/classes identified:	1- Player – provides the input to block an attack. 2- Character – blocks attacks or receives damage if the attack is not blocked. 3- Enemy – attacks the character. 4- Attack – determines the damage dealt to the character. 5- Health – keeps track of the character's health. 6- Input/Keyboard – detects the block key.
Possible responseabilities	1- Player → provides input to block. 2- Input/Keyboard → detects when the block key is pressed. 3- Enemy → performs an attack. 4- Attack → deals damage if the attack is not blocked. 5- Character → blocks the attack or receives damage. 6- Health → decreases when the character receives damage.

Use case 5	Character Dies
Actor(s):	Player
Trigger:	The character's health reaches zero.
Happy Flow:	1 – The character receives damage. 2 – The character's health reaches zero. 3 – The game detects that the character has died. 4 – A game-over screen is displayed. 5 – The player chooses to restart the game. 6 – The game restarts.
Alternative	1 – The character's health reaches zero. 2 – The game-over screen is displayed. 3 – The player chooses to quit the game. 4 – The game closes.
Objects/classes identified:	1- Player – chooses whether to restart or quit the game. 2- Character – dies when its health reaches zero. 3- Health – keeps track of the character's health. 4- Game – handles the character's death and the restart or quit options. 5- GameOverScreen – displays the game-over screen and available options.
Possible responseabilities	1- Player → chooses to restart or quit the game. 2- Health → keeps track of the character's health. 3- Character → dies when health reaches zero. 4- Game → detects the character's death and handles the game state.

	5- GameOverScreen → displays the restart and quit options.
--	--

4 Class Responsibilities and Relationships

Class	Responsibility	Collaboration
Player	Provide input, control the character, perform actions, block attacks, choose to restart or quit	Input/Keyboard, Character, Game, GameOverScreen
Input/Keyboard	Detect start, movement, action, and block inputs	Player, Game, Character
Game	Start the game, process player input, control the game state, detect when the character dies, and handle restarting or quitting	Player, Input/Keyboard, Character, StartupScreen, GameOverScreen
Character	Move, perform actions, block attacks, receive damage, and die when health reaches zero	Player, Game, Enemy, Attack, Health
StartupScreen	Display the startup screen before the game begins	Game, Player
Enemy	Perform attacks against the character	Attack, Character
Attack	Determine whether an attack is blocked and deal damage when it is not blocked	Enemy, Character, Health
Health	Keep track of the character's health and decrease it when damage is received	Character, Attack
GameOverScreen	Display the game-over screen and the options to restart or quit	Game, Player